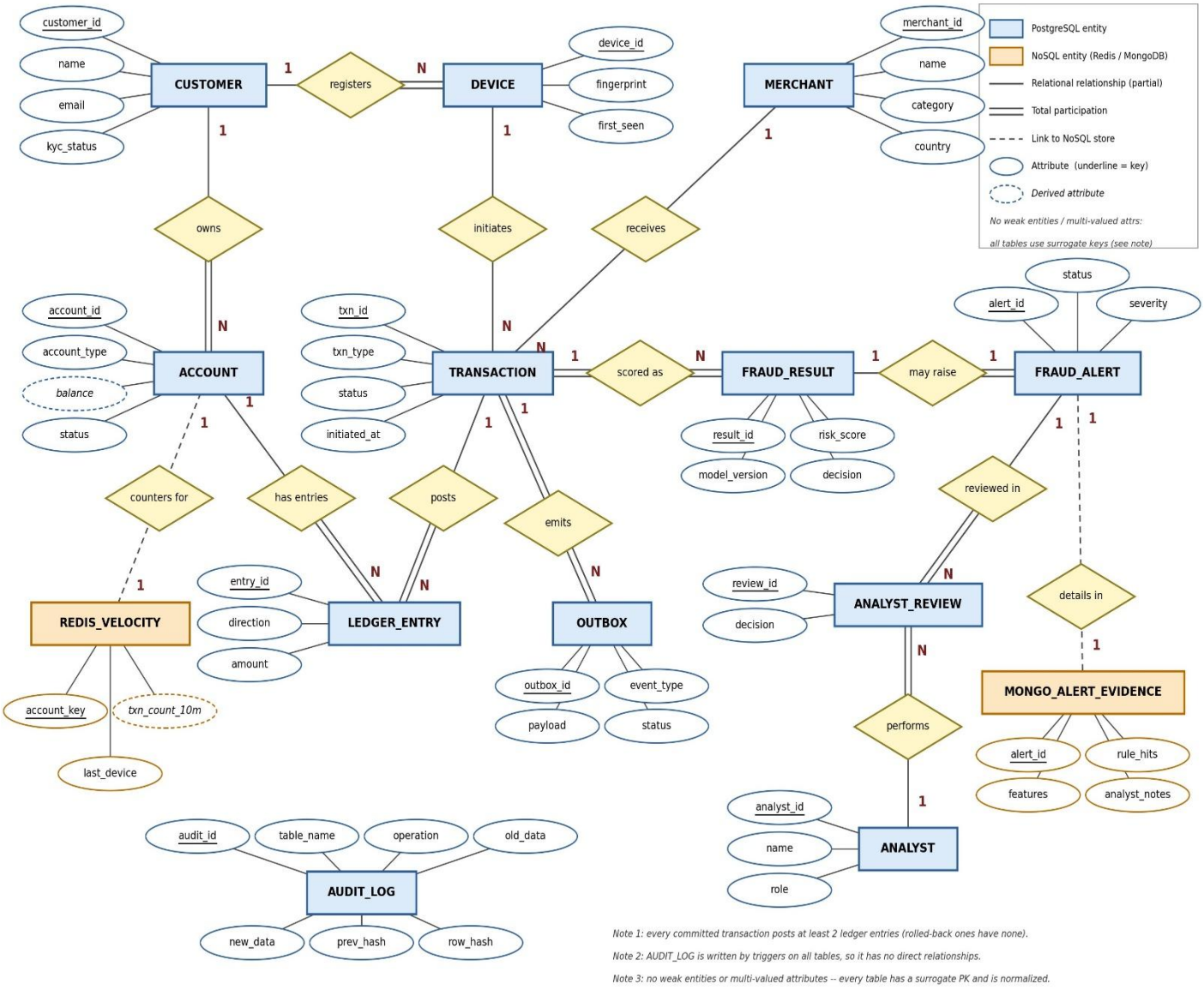
Financial Transaction Ledger with Real-Time Fraud Triage

– ER Diagram & Schema

1. Entities

Entity	Description	Primary Key
Customer	Account holder who owns accounts and registers devices.	customer_id
Device	Device used by a customer to initiate transactions.	device_id
Merchant	Business that receives payment transactions.	merchant_id
Account	Bank account holding a cached balance, owned by one customer.	account_id
Transaction	A single payment or transfer event.	txn_id
Ledger_Entry	Debit or credit line posted against an account for a transaction.	entry_id
Fraud_Result	ML risk score (Isolation Forest) computed for a transaction.	result_id
Fraud_Alert	Alert raised when a fraud result crosses the review threshold.	alert_id
Analyst	Staff member who reviews fraud alerts.	analyst_id
Analyst_Review	An analyst's decision on a specific alert.	review_id
Outbox	Queued event for syncing ledger changes to the NoSQL stores.	outbox_id
Audit_Log	Tamper-evident, trigger-populated history of all table changes.	audit_id
Redis_Velocity (NoSQL)	Real-time per-account transaction-velocity counters, held in Redis.	account_key *
Mongo_Alert_Evidence (NoSQL)	Rule hits, feature values and analyst notes behind an alert, held in MongoDB.	alert_id *

2. ER Diagram:



- Dashed lines link PostgreSQL entities to the two NoSQL stores (Redis, MongoDB);
- double lines mark total participation
- No weak entities, multi-valued attributes or n-ary relationships are present — every table uses a surrogate primary key and the schema is fully normalized (see Section 3).

3. Relationships

All thirteen relationships in the diagram are binary. Ten are one-to-many, two are one-to-one, and none are many-to-many — so no junction tables are required anywhere in this schema. Total participation means every instance of that entity must take part in the relationship.

Relationship	Entities	Cardinality	Participation
Registers	Customer – Device	1 : N	Device total; Customer partial
owns	Customer – Account	1 : N	Account total; Customer partial
initiates	Device – Transaction	1 : N	Both partial
receives	Merchant – Transaction	1 : N	Both partial
scored as	Transaction – Fraud_Result	1 : N	Both total
may raise	Fraud_Result – Fraud_Alert	1 : 1	Fraud_Alert total; Fraud_Result partial
reviewed in	Fraud_Alert – Analyst_Review	1 : N	Analyst_Review total; Fraud_Alert partial
performs	Analyst – Analyst_Review	1 : N	Analyst_Review total; Analyst partial
has entries	Account – Ledger_Entry	1 : N	Ledger_Entry total; Account partial
posts	Transaction – Ledger_Entry	1 : N	Ledger_Entry total; Transaction partial
emits	Transaction – Outbox	1 : N	Both total
counters for *	Account – Redis_Velocity	1 : 1	Redis_Velocity total; Account partial
details in *	Fraud_Alert – Mongo_Alert_Evidence	1 : 1	Both total

* These two links go to NoSQL stores via the transactional outbox worker, not an enforced foreign key — see Section 5 for the consistency mechanism. Audit_Log is populated by triggers on every table above, so it has no modeled relationship of its own.

4. Table Definitions

Customer

Column	Data Type	Constraints
customer_id	INT	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	NOT NULL, UNIQUE
kyc_status	VARCHAR(20)	CHECK (kyc_status IN ('PENDING','VERIFIED','REJECTED'))

Device

Column	Data Type	Constraints
device_id	INT	PRIMARY KEY
customer_id	INT	FOREIGN KEY -> Customer(customer_id), NOT NULL
fingerprint	VARCHAR(128)	NOT NULL, UNIQUE
first_seen	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Merchant

Column	Data Type	Constraints
merchant_id	INT	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
category	VARCHAR(50)	—
country	VARCHAR(50)	—

Account

Column	Data Type	Constraints
account_id	INT	PRIMARY KEY
customer_id	INT	FOREIGN KEY -> Customer(customer_id), NOT NULL
account_type	VARCHAR(20)	CHECK (account_type IN ('SAVINGS','CURRENT','WALLET'))
balance	DECIMAL(14,2)	NOT NULL, DEFAULT 0, CHECK (balance >= 0)
status	VARCHAR(20)	CHECK (status IN ('ACTIVE','FROZEN','CLOSED'))

balance is conceptually derived (SUM of Ledger_Entry amounts) but physically cached here for read performance; a scheduled reconciliation query checks the two stay in agreement.

Transaction

Column	Data Type	Constraints
txn_id	INT	PRIMARY KEY
device_id	INT	FOREIGN KEY -> Device(device_id)
merchant_id	INT	FOREIGN KEY -> Merchant(merchant_id)
txn_type	VARCHAR(20)	CHECK (txn_type IN ('TRANSFER','PAYMENT','WITHDRAWAL','DEPOSIT'))
status	VARCHAR(20)	NOT NULL, CHECK (status IN ('PENDING','COMMITTED','ROLLED_BACK'))
initiated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP

device_id and merchant_id are nullable: an internal transfer between a customer's own accounts has no merchant, and some channels do not report a device.

Ledger_Entry

Column	Data Type	Constraints
entry_id	INT	PRIMARY KEY
account_id	INT	FOREIGN KEY -> Account(account_id), NOT NULL
txn_id	INT	FOREIGN KEY -> Transaction(txn_id), NOT NULL
direction	VARCHAR(6)	NOT NULL, CHECK (direction IN ('DEBIT','CREDIT'))
amount	DECIMAL(14,2)	NOT NULL, CHECK (amount > 0)

A deferred trigger on Transaction enforces SUM(debit) = SUM(credit) per txn_id at commit time. A rolled-back transaction posts zero ledger entries.

Fraud_Result

Column	Data Type	Constraints
result_id	INT	PRIMARY KEY
txn_id	INT	FOREIGN KEY -> Transaction(txn_id), NOT NULL, UNIQUE
risk_score	INT	NOT NULL, CHECK (risk_score BETWEEN 0 AND 100)
model_version	VARCHAR(20)	NOT NULL
decision	VARCHAR(20)	NOT NULL, CHECK (decision IN ('APPROVE','REVIEW','ROLLBACK'))

Fraud_Alert

Column	Data Type	Constraints
alert_id	INT	PRIMARY KEY
result_id	INT	FOREIGN KEY -> Fraud_Result(result_id), NOT NULL, UNIQUE
severity	VARCHAR(10)	CHECK (severity IN ('LOW','MEDIUM','HIGH'))
status	VARCHAR(20)	DEFAULT 'OPEN', CHECK (status IN ('OPEN','REVIEWED','DISMISSED'))

Analyst

Column	Data Type	Constraints
analyst_id	INT	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
role	VARCHAR(30)	—

Analyst_Review

Column	Data Type	Constraints
review_id	INT	PRIMARY KEY
alert_id	INT	FOREIGN KEY -> Fraud_Alert(alert_id), NOT NULL
analyst_id	INT	FOREIGN KEY -> Analyst(analyst_id), NOT NULL
decision	VARCHAR(20)	CHECK (decision IN ('CONFIRM_FRAUD','FALSE_POSITIVE','ESCALATE'))

Free-text review notes are kept in Mongo_Alert_Evidence, not duplicated here.

Outbox

Column	Data Type	Constraints
outbox_id	INT	PRIMARY KEY
txn_id	INT	FOREIGN KEY -> Transaction(txn_id), NOT NULL
event_type	VARCHAR(30)	NOT NULL
payload	JSONB	NOT NULL
status	VARCHAR(20)	DEFAULT 'PENDING', CHECK (status IN ('PENDING','SENT','FAILED'))

Written inside the same transaction as the ledger entries; a background worker reads pending rows and syncs Redis_Velocity and Mongo_Alert_Evidence, then marks them SENT.

Audit_Log

Column	Data Type	Constraints
audit_id	INT	PRIMARY KEY
table_name	VARCHAR(50)	NOT NULL
operation	VARCHAR(10)	CHECK (operation IN ('INSERT','UPDATE','DELETE'))
old_data	JSONB	—
new_data	JSONB	—
prev_hash	CHAR(64)	—
row_hash	CHAR(64)	NOT NULL

Populated entirely by triggers on the tables above; row_hash chains to prev_hash so tampering breaks the sequence. Append-only via restricted privileges.

NoSQL stores (not relational tables)

Store	Key / Document shape	Purpose
Redis_Velocity (Redis, key-value)	Key: velocity:{account_id} → { txn_count_10m: INT, last_device: STRING }, TTL 10 min	Sub-millisecond velocity lookup during fraud scoring, instead of a SQL window query on Ledger_Entry.
Mongo_Alert_Evidence (MongoDB, document)	_id: alert_id → { rule_hits: [...], features: {...}, analyst_notes: STRING }	Flexible, per-alert evidence that differs alert to alert; avoids a wide, sparse relational table.

PostgreSQL remains the source of truth for both. Each document is rebuildable from the Outbox and the relational tables, so neither NoSQL store holds data that cannot be reconstructed.

5. Foreign Key Summary :

Table	Foreign Key	References	Relationship
Device	customer_id	Customer(customer_id)	registers
Account	customer_id	Customer(customer_id)	owns
Transaction	device_id	Device(device_id)	initiates
Transaction	merchant_id	Merchant(merchant_id)	receives
Fraud_Result	txn_id	Transaction(txn_id)	scored as
Fraud_Alert	result_id	Fraud_Result(result_id)	may raise
Analyst_Review	alert_id	Fraud_Alert(alert_id)	reviewed in
Analyst_Review	analyst_id	Analyst(analyst_id)	performs
Ledger_Entry	account_id	Account(account_id)	has entries
Ledger_Entry	txn_id	Transaction(txn_id)	posts
Outbox	txn_id	Transaction(txn_id)	emits

Audit_Log carries no foreign key, since it is populated by triggers on every table above rather than participating in a modeled relationship. Redis_Velocity and Mongo_Alert_Evidence are linked by application-level key (account_id, alert_id respectively), not an enforced relational foreign key, since they live outside PostgreSQL.

6. Mapping the ER Diagram to the Relational Model

The standard 7-step ER-to-relational algorithm is applied below. Four of the seven steps do not apply to this design (no weak entities, no M:N relationships, no multi-valued attributes, no n-ary relationships) — each is still shown, with the reason, rather than skipped silently.

Step 1: Mapping of Regular (Strong) Entity Types

Rule: Create a relation for each strong entity, including all simple attributes, and assign a primary key.

Applies to our project: All 12 relational entities are strong — every one carries its own surrogate primary key, so each maps directly to a relation with no foreign keys yet.

Schema after this step:

```
CUSTOMER(customer_id, name, email, kyc_status)
DEVICE(device_id, fingerprint, first_seen)
MERCHANT(merchant_id, name, category, country)
ACCOUNT(account_id, account_type, balance, status)
TRANSACTION(txn_id, txn_type, status, initiated_at)
FRAUD_RESULT(result_id, risk_score, model_version, decision)
FRAUD_ALERT(alert_id, severity, status)
ANALYST(analyst_id, name, role)
ANALYST_REVIEW(review_id, decision)
LEDGER_ENTRY(entry_id, direction, amount)
OUTBOX(outbox_id, event_type, payload, status)
AUDIT_LOG(audit_id, table_name, operation, old_data,
new_data, prev_hash, row_hash)
```

Step 2: Mapping of Weak Entity Types

Rule: Create a relation for the weak entity including its simple attributes, add the owner's primary key as a foreign key, and form a composite primary key.

Applies to our project: Not applicable. Every entity in the diagram already has its own surrogate primary key (entry_id, outbox_id, result_id, etc.), so none of them need an owner's key to be identified. There are no weak entities in this design.

No change to the schema at this step.

Step 3: Mapping of Binary 1:1 Relationship Types

Rule: Place the primary key of one entity as a foreign key in the other relation, preferably on the side with total participation.

Applies to our project: Only one 1:1 relationship exists — ‘may raise’ between FRAUD_RESULT and FRAUD_ALERT. FRAUD_ALERT has total participation (an alert cannot exist without a result), so its FK is placed there.

Schema after this step:

```
FRAUD_ALERT(alert_id, result_id FK → FRAUD_RESULT,
severity, status)
```

Step 4: Mapping of Binary 1:N (One-to-Many) Relationship Types

Rule: Place the primary key of the 1-side entity as a foreign key in the N-side relation, along with any relationship attributes.

Applies to our project: Nine of the thirteen relationships are 1:N. Each places the 1-side's key on the N-side. LEDGER_ENTRY is the N-side of two separate 1:N relationships (‘has entries’ and ‘posts’), so it picks up two foreign keys, not one.

Schema after this step:

```
DEVICE(device_id, customer_id FK → CUSTOMER,
fingerprint, first_seen)
ACCOUNT(account_id, customer_id FK → CUSTOMER,
account_type, balance, status) TRANSACTION(txn_id,
device_id FK → DEVICE,             merchant_id FK →
MERCHANT, txn_type,             status, initiated_at)
FRAUD_RESULT(result_id, txn_id FK → TRANSACTION,
risk_score, model_version, decision)
ANALYST_REVIEW(review_id, alert_id FK → FRAUD_ALERT,
analyst_id FK → ANALYST, decision)
LEDGER_ENTRY(entry_id, account_id FK → ACCOUNT,
txn_id FK → TRANSACTION, direction, amount)
OUTBOX(outbox_id, txn_id FK → TRANSACTION,
event_type, payload, status)
```

Step 5: Mapping of Binary M:N (Many-to-Many) Relationship Types

Rule: Create a new junction table containing the primary keys of both participating entities as a composite primary key and foreign keys, plus any relationship attributes.

Applies to our project: Not applicable. There is no many-to-many relationship in the diagram. The closest candidate — alerts and analysts — was deliberately modeled as ANALYST_REVIEW, a regular entity with its own key and a decision attribute, rather than a bare M:N diamond, so it was already handled under Step 4.

No change to the schema at this step.

Step 6: Mapping of Multivalued Attributes

Rule: Create a new relation for the multivalued attribute containing the attribute itself and the entity's primary key as a foreign key, to form a composite primary key.

Applies to our project: Not applicable. Normalization already pushed anything that could have been multivalued (e.g. multiple devices per customer) into its own entity (DEVICE), so no attribute in the diagram holds more than one value.

No change to the schema at this step.

Step 7: Mapping of N-ary Relationship Types

Rule: Create a new relation for relationships involving three or more entities, using the primary keys of all participating entities as foreign keys and a composite primary key.

Applies to our project: Not applicable. Every relationship diamond in the diagram connects exactly two entities, so nothing in the design needs three or more entities tied together at once.

No change to the schema at this step.

7. Final Relational Model

All 12 relations after applying the 7 steps above. Each box lists every attribute; the primary key is underlined and tagged P.K, and each foreign key is italicized and tagged F.K, with an arrow drawn from the foreign key to the primary key it references. Redis_Velocity and Mongo_Alert_Evidence are not relations — they sit outside this mapping.

